TECHNICAL DOCUMENT

Boundra

Autonomy within your boundaries

A risk-aligned agent wallet: an AI agent manages funds for an owner, and the owner's limits are enforced by a smart contract, not by trust.

Event	FinTechathon 2026, Topic F
Network	Base Sepolia testnet (chain 84532) with MockUSDC and mock vaults
Version	Repository `main` at commit `0593c8a`, 20 October 2026
Team	Person 1 (contracts, chain) · Person 2 (backend, ACCE, data) Person 3 (frontend, delivery, QA)

Contents

1 Overview 3
- 1.1 The problem 3
- 1.2 The idea 3
- 1.3 Design principles 4
- 1.4 Boundra at a glance 4

2 System architecture 5
- 2.1 Who does what 6
- 2.2 Repository layout 6

3 Smart contracts 8
- 3.1 The contracts 8
- 3.2 The mandate 8
- 3.3 How value moves 9
- 3.4 Rules the executor enforces 9
- 3.5 Events used as evidence 10
- 3.6 The Base Sepolia deployment 10

4 Clarification: ACCE 12
- 4.1 What the owner is asked about 12
- 4.2 The algorithm 12
- 4.3 Worked examples 13

5 From answers to an authorized mandate 15
- 5.1 The draft 15
- 5.2 Authorization, step by step 15
- 5.3 State machines 16
- 5.4 Choosing the agent 16

6 The agent: worker 18
- 6.1 Three modes 18
- 6.2 How work is scheduled 18
- 6.3 Deciding what to do 19
- 6.4 Executing an action safely 19
- 6.5 Watching for risk 20

7 Data model 22
- 7.1 Tables 22
- 7.2 Concurrency rules 23

8 API and sessions 24
- 8.1 Signing in with Ethereum 24
- 8.2 API tokens for apps and assistants 25
- 8.3 The owner journey as API calls 25
- 8.4 Errors and health 25

9 Web app 27
- 9.1 Pages 27

9.2 Wallet transactions done honestly 27
9.3 Reviewing a mandate 28
9.4 Design 28

10 Language model use 29
10.1 Providers 29
10.2 Validation and fallback 29

11 SDK and MCP server 31
11.1 `@boundra/sdk` 31
11.2 MCP server 32

12 Security model 33
12.1 Trust boundaries 33
12.2 Threats and mitigations 33
12.3 Contract evidence 34
12.4 Operational rules 34

13 Testing and evidence 35
13.1 Test suites 35
13.2 The end-to-end journey 35
13.3 Benchmark: B0–B3 36

14 Deployment and operations 37
14.1 Docker Compose stack 37
14.2 Render, free plan 37
14.3 Settings that matter 38
14.4 Checking a deployment 39

15 Limitations and roadmap 40
15.1 Known limitations 40
15.2 Roadmap 40

A Glossary 41

B Where to find things 42

CHAPTER 1

Overview

How to read this document. Chapters 2–11 follow the system from the chain upwards: contracts, clarification, mandates, the agent, data, API, web app, language model and integrations. Chapter 12 collects the security model, chapter 13 the evidence and chapter 14 how it runs. The appendix has a glossary and a map of where each topic lives in the repository.

1.1 The problem

People increasingly want software agents, including AI agents, to act on their money: move savings into a vault, pull it out when a risk appears, keep it working while they are busy. Today that usually means one of two bad choices. Either the agent gets a key that can do *anything* with the funds, so the owner has to trust it completely, or every action waits for the owner's manual approval, so it is not really an agent.

There is a second, quieter problem: the agent often does not know what the owner actually wants. "Keep my money safe but earn something" says nothing about whether the owner can live without the money for a month. An agent that guesses can lock funds the owner needs next week.

1.2 The idea

Boundra solves both problems with two pieces that work together:

1. **Ask only what changes the decision.** Before anything is set up, Boundra asks the owner a few clarifying questions, chosen by an algorithm called ACCE (Action-Conditioned Counterfactual Elicitation). It asks a question only if the answer could change which action the agent is allowed to take, and it stops as soon as no answer could.
2. **Turn the answers into limits a contract enforces.** The answers become a *mandate*: an exact set of limits (which vaults, how much per action, how much in total, how long funds may be locked, until when). The owner reviews it in plain language, approves it, and signs it into their own *executor* contract on the blockchain. From then on the agent can only do what the mandate allows, and the contract rejects everything else.

The guarantee in one sentence

Even if the agent's key is stolen or its model is wrong, the worst it can do is bounded by the mandate the owner signed, withdrawals always pay the owner, and the owner can pause or revoke at any time.

1.3 Design principles

Principle	What it means in the implementation
No owner keys, ever	The server never holds or asks for the owner's private key. Every owner action (create executor, set mandate, fund, pause, revoke, recover) is a transaction the owner signs in their own wallet.
The contract is the boundary	Limits are checked by `BoundraExecutor` on chain. The backend checks them too, but only to fail early and explain; it is never the last line of defence.
Authority is deterministic	Which questions are asked, which vault is selected and which limits are drafted are decided by deterministic code. A language model may only reword text, and its output is validated with a template fallback.
Chain state is the truth	PostgreSQL stores workflow and history. Before every action the worker re-reads confirmed chain state and reconciles; a database row is never taken as proof of authority, funding or execution.
Honest evidence	No invented addresses, transaction hashes or benchmark results. Activity entries link to the block explorer only for real transactions; holds and refusals carry no hash.
Testnet only	Base Sepolia with MockUSDC and two controlled mock vaults. Nothing here touches real funds before an independent audit.

1.4 Boundra at a glance

Part	Technology	Role
Smart contracts	Solidity 0.8.30, Foundry	MockUSDC, faucet, two mock vaults, factory, one executor per owner.
Web app	Next.js 16, React 19, wagmi 3, viem 2.57	Owner journey, wallet transactions, risk admin page, API tokens.
API	NestJS 11, Zod, Prisma 7	Sign-in with Ethereum, intents and ACCE, mandate drafts, dashboard, activity.
Worker	NestJS, viem	Chain monitor, decision policy, bounded execution with crash recovery.
Database	PostgreSQL 17	Workflow, decisions, transaction attempts, jobs, activity.
ACCE	TypeScript package	Question selection, vault selection, mandate builder.
Language model	OpenAI or Anthropic (optional)	Rewording questions and summaries only, always validated.
SDK and MCP server	TypeScript, MCP SDK 1.32	Let other apps, AI assistants and owners' own agents use Boundra within the same limits.
Deployment	Docker Compose with Caddy, or Render	Reproducible stack with HTTPS, backups and health checks.

CHAPTER 2

System architecture

Boundra has four running parts (web app, API, worker, database) and a set of contracts on Base Sepolia. The owner's wallet talks to the contracts directly; the backend never signs for the owner. Figure 2.1 shows how they connect.

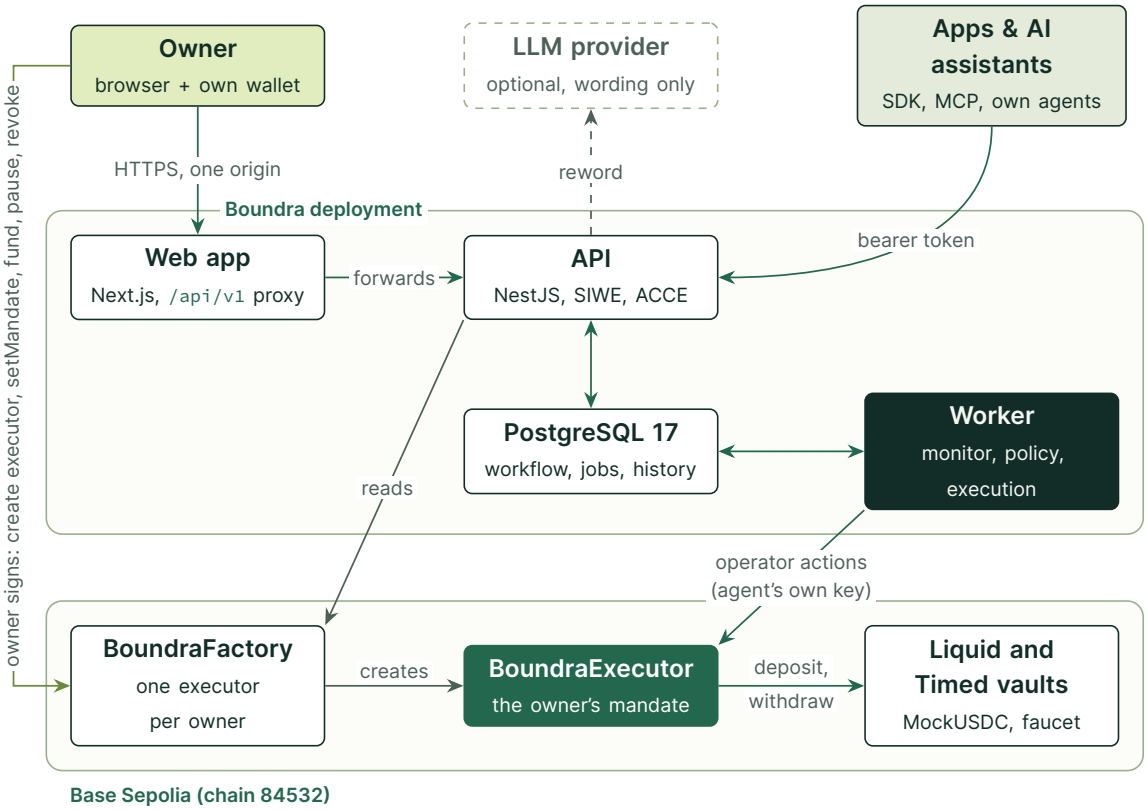


Figure 2.1: The parts of Boundra and how they talk. Green arrows carry authority; grey arrows only read.

2.1 Who does what

Part	Does	Never does
Owner's wallet	Signs sign-in messages and every owner transaction: create executor, set mandate, approve and fund, pause, resume, revoke, recover.	Hands its key to anyone.
Web app	Guides the owner, prepares the exact transactions, shows chain-confirmed state, forwards `/api/v1/*` to the API so the browser sees one origin.	Hold keys or secrets.
API	Sessions, intents and ACCE questions, mandate drafts and previews, dashboard and activity from confirmed state.	Sign transactions or move funds.
Worker	Watches vault risk, decides what the agent should do, and executes bounded operator transactions with the agent's own key.	Act outside a confirmed mandate, or for a mandate naming another agent.
Executor contract	Enforces the mandate on every operator call; pays withdrawals to the owner.	Accept arbitrary calls, recipients or upgrades.

One origin. The browser only ever talks to the web app's origin. Sessions use a first-party `__Host-` cookie, and every state-changing request must carry that exact `Origin`. The web app's route handler forwards `/api/v1/*` to the API (in a public Compose deployment, Caddy routes it directly), so the cookie never crosses sites.

2.2 Repository layout

The code is one pnpm workspace (Node.js 22.22, pnpm 10.32). Each team member owns parts of it.

Path	Contents
`contracts/`	Solidity contracts, Foundry tests and the deploy script (Person 1).
`packages/chain`	Generated ABIs, the deployment-manifest schema, `BoundraChainReader` and `prepareAction`.
`packages/shared`	Zod schemas, exact amount types, stable error codes.
`packages/database`	Prisma schema, migrations, client and the job queue.
`packages/acce`	The ACCE planner, vault selection and the mandate builder.
`packages/llm`	Optional language-model providers with validation and fallbacks.
`packages/sdk`	`@boundra/sdk`: owner API client and bounded operator.
`apps/api`	NestJS REST API with OpenAPI.
`apps/worker`	Chain boundary (`chain/`), decisions and jobs (`orchestration/`), stores (`persistence/`).

Path	Contents
`apps/web`	Next.js owner app and landing page.
`apps/mcp`	MCP server for AI assistants.
`evaluation/`	B0–B3 benchmark runner and persona fixtures.
`deploy/`, `compose.yaml`, `render.yaml`	Deployment: Compose stack, Caddy, backups, manifests, Render.

CHAPTER 3

Smart contracts

The contracts are the part that makes the guarantee real. They are small on purpose (about 460 lines of Solidity 0.8.30 in total) and have no admin override, upgrade path or arbitrary-call entry point.

3.1 The contracts

Contract	Role
`MockUSDC`	Test stablecoin with 6 decimals and a hard supply cap of 1,000,000. Only the faucet can mint.
`MockUSDCFaucet`	Gives 1,000 MockUSDC per claim, at most once a day per address.
`LiquidVault`	Mock vault with no lockup. Illustrative yield 300 bps a year (metadata only: principal stays 1:1).
`TimedVault`	Mock vault with a real 30-day lockup. Illustrative yield 600 bps a year.
`BoundraFactory`	Fixes the reviewed token and the two vaults, and creates one executor per owner (`createExecutor`, `executorOf`).
`BoundraExecutor`	One per owner. Holds the owner's funds, stores the mandate, and is the only thing the agent (the *operator*) can call.

Both vaults have a *risk state* set by the risk admin: `NORMAL` (0) or `DEGRADED` (1), plus a global withdrawal capacity. A degraded vault refuses new deposits. This is a controlled demo fixture that stands in for a real protocol incident, not an oracle.

3.2 The mandate

A mandate is a fixed tuple of twelve fields. The owner calls `setMandate(tuple)` on their executor; the contract records `keccak256(abi.encode(tuple))` as the mandate hash and emits `MandateSet(version, mandateHash, operator)`.

Field	Type	Meaning
`chainId`	uint256	Must be 84532 (Base Sepolia).
`asset`	address	The reviewed MockUSDC.
`operator`	address	The agent allowed to act. Boundra's agent or the owner's own.
`beneficiary`	address	Who receives withdrawals. Must equal the owner.

Field	Type	Meaning
`expiresAt`	uint64	When the mandate stops (seconds).
`allowedVaults`	bitmask	1 Liquid, 2 Timed, 3 both.
`allowedActions`	bitmask	1 deposit, 2 withdraw, 3 both.
`maxLockupSeconds`	uint32	Longest lockup the owner accepted.
`maxDepositPerAction`	uint256	Cap per deposit.
`maxCumulativeDeposits`	uint256	Total deposit budget for this mandate.
`maxWithdrawPerAction`	uint256	Cap per withdrawal.
`allocationTarget`	uint256	Amount the single allocation aims for.

3.3 How value moves

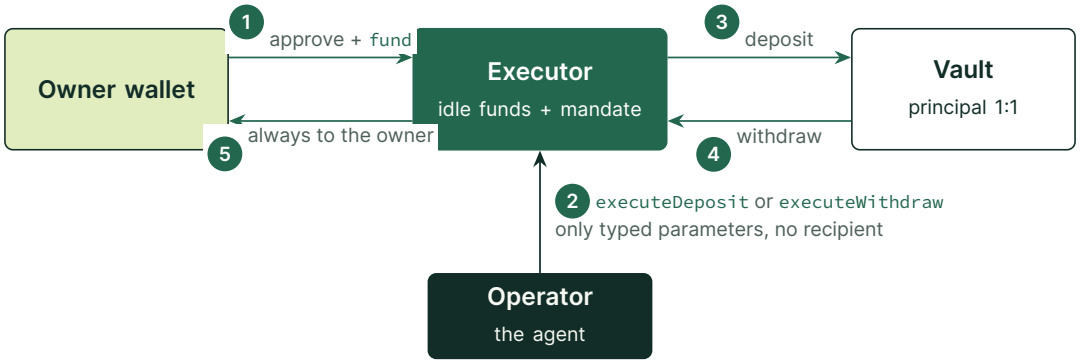


Figure 3.1: The operator can only ask the executor to deposit or withdraw. Funds leave the system only to the owner.

3.4 Rules the executor enforces

Every operator call has one of two fixed shapes, with no recipient argument and no arbitrary calldata:

```
executeDeposit (actionId, version, nonce, deadline, target, amount)
executeWithdraw(actionId, version, nonce, deadline, target, amount)
```

On each call the executor checks:

- the caller is the mandate's operator, the mandate is not paused, revoked or expired, and the `version` is current;
- the `nonce` equals `nextActionNonce` and the `actionId` was never used (neither ever resets, even across mandates; a reverted call consumes neither);
- the `deadline` is in the future (exclusive) and not after the mandate's expiry;
- the target vault and the action are allowed, and the amount is within the per-action cap, the remaining budget and the allocation target;
- a deposit does not go into a degraded vault, and the lockup vault is only used if the mandate accepted its 30-day lockup and expires more than 30 days plus a one-day buffer after the deposit;

- there is *exactly one* deposit allocation per mandate: a cap-limited partial deposit completes it, and the leftover stays idle and recoverable by the owner;
- a withdrawal pays the owner directly and never refills the deposit budget.

Owner controls survive everything: `pause`, `resume` (only if not expired or revoked), `revoke`, `recoverIdle(amount)` and `ownerWithdraw(target, amount)`. Recovery still respects the vault's real lockup and capacity. A new mandate can replace the old one only when both vault positions are zero, so a fresh deposit budget always needs the owner's explicit authorization after settlement.

> **What the contract cannot protect.** Within the mandate, a compromised operator can still act at inconvenient times or use the remaining budget in otherwise allowed ways. Caps limit damage; they do not prove that an action suits the owner financially. Already executed transactions are not undone by pause or revoke.

3.5 Events used as evidence

Event	Used for
`ExecutorCreated(owner, executor)`	Registering the owner's executor (the API re-reads `executorOf`).
`MandateSet(version, mandateHash, operator)`	Matching the confirmed mandate to the approved preview.
`ActionExecuted(actionId, version, nonce, action, target, amount)`	Proof that an agent action really happened. A broadcast hash alone is not proof.
`RiskStateChanged(level, capacity)`	The worker's risk monitor, only from the two pinned vault addresses.
`Deposited`, `Withdrawn`	Vault-side position changes.

3.6 The Base Sepolia deployment

Person 1 deployed the contracts with `contracts/script/DeployBoundra.s.sol` at block 48,322,785. Before it was pinned in `deploy/manifests/base-sepolia.json`, every role was read back from the contracts themselves and every contract's deployed bytecode was compared with the repository's compiled code. Because anyone can deploy look-alike contracts, the API and worker trust only this manifest: they check the chain, the addresses and the keccak256 hash of

each contract's runtime code before acting.

Contract	Address
Factory	`0xFbD91368BfFA9AF5C7034bCB7F53Cc1eC906b7f4`
MockUSDC	`0x13e7a4d02941a029Cf841f16E38eF377cb21e1Cf`
Faucet	`0xEbdc09B132066fF067E30FE077A58E818801FA2f`
LiquidVault	`0x7C0fC1a0551d5099d093E8D8b34a214758f7B27a`
TimedVault	`0x7ffD08b7b52E26023142060D3f5dBDe27757CD84`
Risk admin	`0xA05F8266B63D8074AcBA1c8f4e7355E7F515F14b`

CHAPTER 4

Clarification: ACCE

ACCE (Action-Conditioned Counterfactual Elicitation) decides which questions to ask the owner. Its rule is simple: *ask a question only if some possible answer would change the action the agent may take*. Everything here is deterministic code in `packages/acce`; a language model can at most reword a question that ACCE has already chosen.

4.1 What the owner is asked about

ACCE works on a small set of *dimensions*. Each has fixed alternatives, an "unsure" option, and ordinal weights from 0 to 3: materiality M (how much it matters), uncertainty U (how unclear it is from the goal alone) and friction (how annoying the question is).

Dimension	Alternatives	M	U	Fric.	Enforced by
`max_lockup_seconds`	no lockup, up to 30 days, unsure	3	3	0	on-chain `maxLockupSeconds`
`withdrawal_latency_seconds`	within a day, not for 30 days, unsure	3	3	1	eligibility excludes lockup vaults
`permitted_vaults`	any reviewed, Liquid only, unsure	1	1	3	on-chain `allowedVaults`
`leverage`	none, leverage OK, unsure	3	1	1	structural: no leveraged vault exists

The amount comes from the owner's goal and the expiry from reviewed defaults; neither is an ACCE question. Unanswered dimensions take the conservative default (no lockup, funds needed soon). The table and the vault descriptors are configuration version `v1`; changing them needs a new version, and never after a held-out benchmark run (chapter 13).

> **Never a silent opt-in.** "Unsure" is always allowed. It resolves to the conservative value and is shown to the owner as *narrowed*. Contradictory answers (accepting a lockup but needing the money within a day) are also narrowed and explained, never silently resolved in the riskier direction.

4.2 The algorithm

1. **Eligibility.** Each deposit candidate (LiquidVault, TimedVault) is checked against what is known, with explicit reasons: not permitted, lockup above tolerance, withdrawal latency above the liquidity need, leverage.

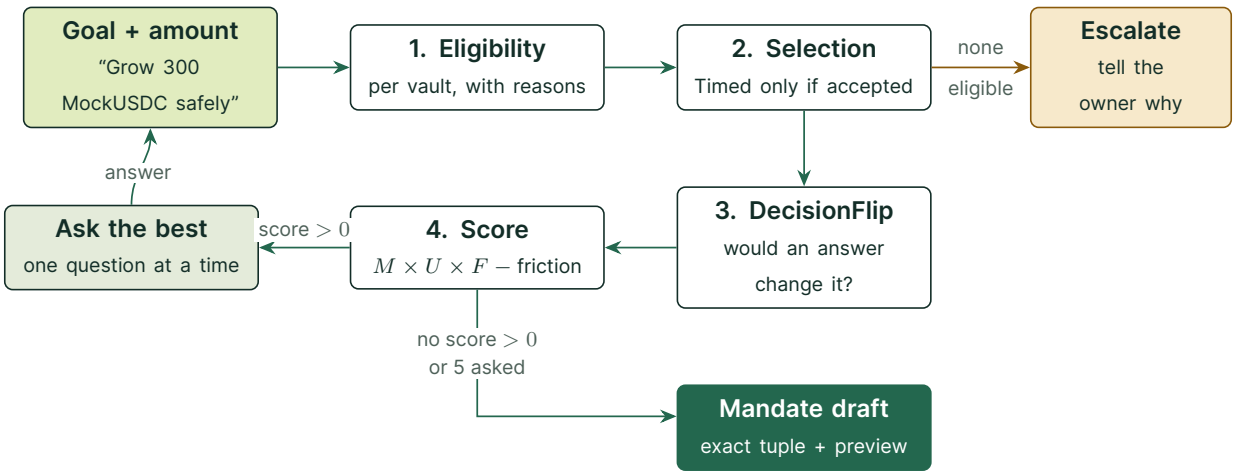


Figure 4.1: ACCE re-plans after every answer and stops as soon as no remaining answer could change the action.

2. **Selection.** TimedVault is selected only if it is eligible *and* the owner explicitly accepted its lockup in exchange for its higher illustrative yield. Otherwise LiquidVault; if nothing is eligible, the intent escalates.
3. **DecisionFlip.** For every unresolved dimension, ACCE builds a truth table over all combinations of answers to the *other* unresolved dimensions and checks how often varying this one changes the selected action. That share maps to a value from 1 to 3; a dimension that never flips the decision gets 0 and is dropped.
4. **Score and stop.** Each remaining dimension scores

$$\text{score} = M \times U \times \text{DecisionFlip} - \text{Friction}.$$

The highest positive score is asked next (ties by dimension order). ACCE stops when no score is positive, or after five questions.

4.3 Worked examples

These outcomes are covered by tests.

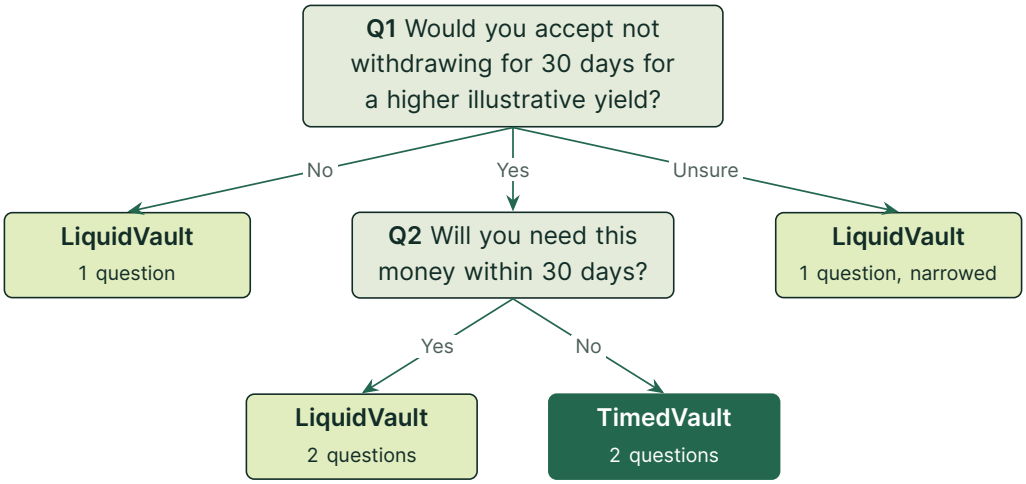


Figure 4.2: Leverage is never asked: no leveraged vault exists, so no answer could change the action.

CHAPTER 5

From answers to an authorized mandate

Once ACCE has a selection, the mandate builder turns it into the exact tuple the executor expects, plus a plain-language preview. Nothing reaches the chain until the owner has reviewed that preview and signed the transactions themselves.

5.1 The draft

The builder (`buildMandate` in `packages/acce`) fills every field deterministically:

- `allowedVaults` contains only the selected vault, and `maxLockupSeconds` equals that vault's lockup (0 for Liquid, 30 days for Timed).
- The deposit cap, the total budget and the allocation target equal the requested amount. If the owner sets a smaller per-action cap, the preview explains that the single allocation will be partial and the rest stays idle.
- The withdrawal cap covers the whole position, so a risk exit can take everything out at once.
- The beneficiary is the owner. The expiry is 7 days by default, or the lockup plus 2 days for TimedVault (at most 60 days).
- The operator is Boundra's agent by default, or an agent the owner names (section 5.4).

The **preview** lists every limit, how it is enforced and the testnet labels, plus a plain summary and notices. Its canonical JSON is hashed (keccak256) into the `previewHash`. Approval must present that exact hash, so the owner can only approve what they were shown; an edit always creates a new revision and supersedes the old one.

5.2 Authorization, step by step

> **Why the worker checks, not the API.** The browser could report anything. The worker reads the executor at the configured confirmation depth (5 blocks), compares the on-chain tuple with the approved revision field by field and checks the mandate hash and funding. Until then the agent does nothing.

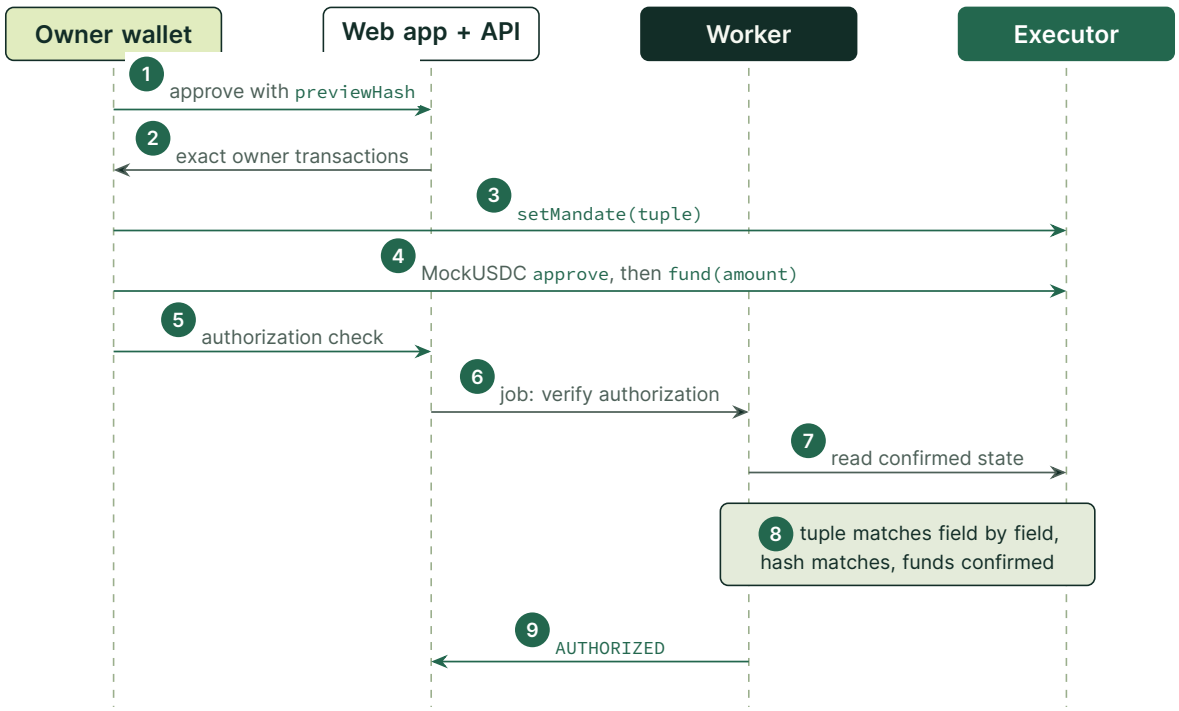


Figure 5.1: Only the worker marks a mandate `AUTHORIZED`, and only from confirmed chain state.

5.3 State machines

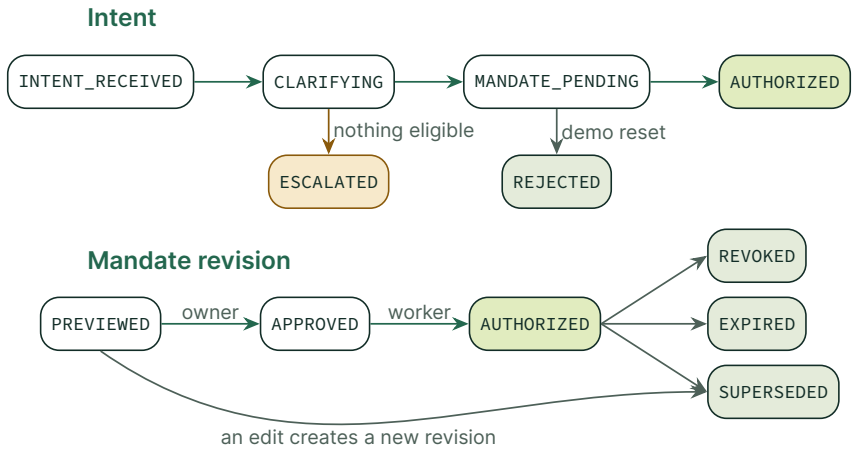


Figure 5.2: At most one mandate per executor can be `AUTHORIZED` (a partial unique index in PostgreSQL).

5.4 Choosing the agent

By default the mandate names Boundra's agent (`OPERATOR_ADDRESS`). An owner who runs their own agent can name its address instead (the `operator` field of a draft). Then:

- the preview carries `operatorSource: "OWNER"` and, as its first notice, says that Boundra does not run or vouch for that agent; both are inside the hashed preview the owner approves;

- the address may not be the owner, their executor, the zero address, a Boundra contract or the risk admin;
- Boundra's worker keeps monitoring the mandate (risk, expiry, escalation) but never transacts for it: it records `HOLD` with reason `OWNER_AGENT_<KIND>`.

The owner's own agent then acts through the SDK or MCP server (chapter 11), bounded by exactly the same contract checks.

CHAPTER 6

The agent: worker

The worker is Boundra's agent. It runs on its own, watches the chain, decides what to do for each authorized mandate, and carries out bounded operator transactions with the agent's own key. It is built so that a crash, a slow RPC node or a chain reorganisation can never make it act twice, act on stale state or lose track of a transaction it signed.

6.1 Three modes

At start-up `runtime/activation.ts` decides how much the worker is allowed to do. It only ever moves up a mode when every check passes.

Mode	When	Behaviour
`observe_only`	No deployment manifest, or activation failed (RPC down, bytecode does not match the pinned hashes).	Heartbeat only; activation is retried periodically.
`monitor_only`	Manifest verified, but signing is not fully configured (`EXECUTION_ENABLED`, operator key, `ATTEMPT_ENCRYPTION_KEY`, gas caps), or the operator equals the risk admin.	Verifies authorizations, monitors risk, records decisions. Never signs.
`execute`	All of the above are present.	Also proposes, signs and reconciles operator actions.

The mode is written into the worker's heartbeat, and the API's readiness endpoint reports it (`executionEnabled` is true only in `execute`).

6.2 How work is scheduled

All work goes through a durable job queue in PostgreSQL (`Job` table). Every poll interval (10 s by default) the scheduler enqueues idempotent jobs: a risk-monitor poll, an evaluation for every authorized mandate, an authorization check for every approved one, and receipt reconciliation for open actions. Job keys contain a time bucket, so two workers enqueueing the same tick create one job.

- Jobs are claimed with `FOR UPDATE SKIP LOCKED` and a lease. If a worker crashes, its job is re-claimed after the lease expires, and the old owner can no longer complete it.
- A workflow change and the jobs it triggers commit in the same database transaction, so no follow-up work is lost between them.

6.3 Deciding what to do

For each authorized mandate the orchestrator takes a fresh snapshot of confirmed chain state and applies a deterministic policy (`orchestration/decision.ts`):

Situation	Decision
Mandate paused	`HOLD`
Position held, vault `NORMAL`	`HOLD` (monitoring)
Position held, vault `DEGRADED`, unlocked, capacity left, withdrawal allowed	`WITHDRAW` min(principal, capacity, per-action cap) to the owner
Position held, `DEGRADED`, but locked, no capacity or withdrawal not allowed	`ESCALATE` to the owner
Allocation already completed (also after an exit)	`HOLD`: never re-deposit
Vault `DEGRADED`, no position	`HOLD`: no new deposit
Idle funds below the planned deposit	`HOLD`: awaiting funding
Mandate names the owner's own agent	`HOLD OWNER_AGENT_<KIND>`: never transact for it
Otherwise	`DEPOSIT` min(target, per-action cap, remaining budget) into the selected vault

Every decision is recorded with its reason, the candidates and the snapshot block (repeated holds are not re-logged). Revoked, expired or replaced mandates change status and stop all decisions.

6.4 Executing an action safely

Executing is where most of the engineering lives. The operator execution service implements the flow below on top of the PostgreSQL execution store; its code is in `chain/execution.ts`.

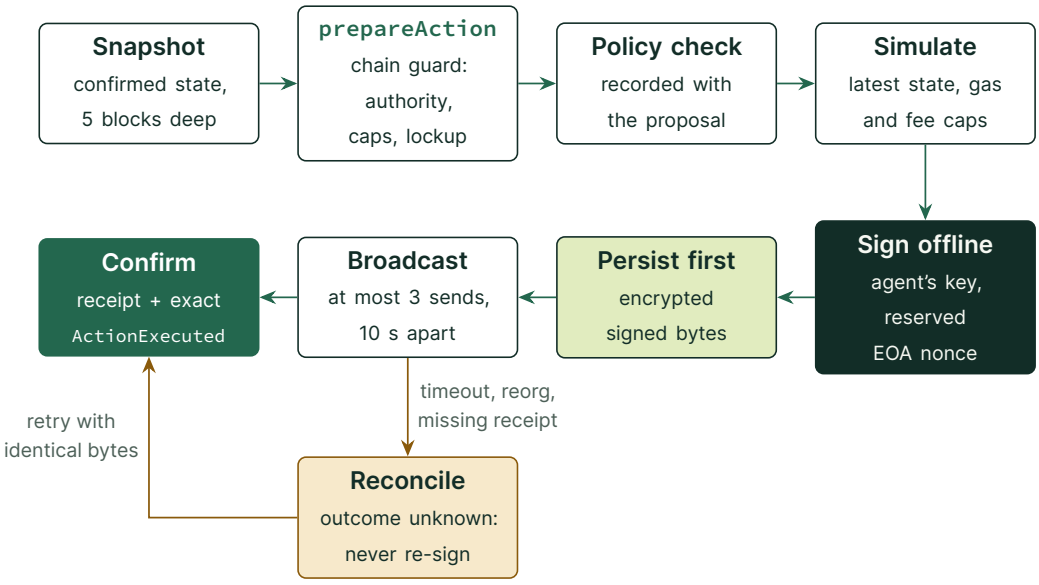


Figure 6.1: The signed transaction is stored before it is sent, so a crash at any point resumes the same transaction.

Leases and fencing. All dispatches for one operator are serialized by a row lease with a monotonic fencing token, held across RPC calls but never as an open database transaction. Every write first checks the token, so a worker that lost its lease cannot commit.

Two kinds of nonce. The executor's *action nonce* and the operator account's *transaction nonce* are different things. Each action reserves its transaction nonce idempotently before signing; the reservation survives restarts.

Persist before broadcast. The signed raw transaction is saved (AES-256-GCM encrypted, with the action ID as associated data) before any network call. After a crash the worker finds it and resumes with the same bytes instead of signing a new transaction.

Ambiguity is never resolved by guessing. If the receipt is missing, the worker asks for the known transaction. Only when both the receipt and the transaction are explicitly absent, and the reserved nonce is still unused, does it re-broadcast, and only the identical bytes, within the persisted budget. Generic RPC errors are not taken as proof of absence.

Releasing a blocked nonce. If policy fails before the action's deadline (for example the owner paused), the old transaction is not resent. Once a confirmed block passes its deadline, the executor is guaranteed to reject it, so the worker broadcasts those identical bytes on purpose: the revert consumes the reserved nonce instead of leaving a gap that would block every later transaction. Release sends share the counter (at most 6 sends in total).

Reorg safety. Terminal receipts are re-checked for at least 20 blocks after inclusion. A reverted attempt keeps its executor action nonce reserved until that window has passed.

State machine	States
`ActionProposal`	`POLICY_CHECKED` → `BROADCAST` → `CONFIRMED` | `FAILED`; `RECONCILE` while unknown; `REJECTED` when the guard refuses before signing.
`TransactionAttempt`	`SIGNED` → `BROADCAST` → `CONFIRMED` | `REVERTED`; `RECONCILE` when ambiguous.

6.5 Watching for risk

`RiskEventMonitor` (`chain/monitor.ts`) scans `RiskStateChanged` logs, but only from the two pinned vault addresses and only at the configured confirmation depth.

- It keeps a durable cursor (next block and the previous block's hash). If that hash changes or the chain goes backwards, it rewinds to the deployment block and rescans, marking affected observations as orphaned.
- Each scan overlaps the previous one by 20 blocks; events are deduplicated by chain, transaction hash and log index.
- Scans cover at most 180 new blocks (`MONITOR_BATCH_BLOCKS`), so each log query spans 199 blocks: the public Base Sepolia RPC rejects ranges above about 200.
- An event never triggers an action directly. It only enqueues an evaluation, which re-reads the vault, position and mandate from the chain.
- Historical risk is recorded separately from the current state, so an old warning does not override

a vault that has recovered.

> **Found in the first live run.** The first run against Base Sepolia showed every monitor poll failing: the worker asked for 520-block log ranges and the public RPC accepts about 200. The batch size is now 180 by default and configurable. On first start the monitor needs roughly 40 minutes to catch up from the deployment block, once.

CHAPTER 7

Data model

PostgreSQL 17 through Prisma 7 (`packages/database/prisma/schema.prisma`). Migrations are applied as a release step (`pnpm db:migrate`) and never reset at start-up.

> **Exact numbers only.** Token amounts and block numbers are stored as `NUMERIC(78,0)`, exact to the full uint256 range; on the wire they are base-unit strings (MockUSDC has 6 decimals), never floating-point numbers. Addresses are stored lowercase and hashes as lowercase `0x` hex.

7.1 Tables

Area	Tables	Purpose and key constraints
Identity	`WalletIdentity`, `AuthChallenge`, `AuthSession`	One row per owner address; single-use SIWE nonces (stored as hashes); sessions and API tokens (`kind` `COOKIE` or `TOKEN`), stored only as hashes.
Clarification	`Intent`, `Question`, `PreferenceRecord`	The goal and exact amount with an optimistic `version`; up to five ordered questions with their scores; versioned answers for audit.
Mandates	`Executor`, `Mandate`	One verified executor per owner; one row per draft revision with the exact tuple, preview, `previewHash`, `mandateHash` and status. At most one `AUTHORIZED` per executor.
Agent	`Decision`, `ActionProposal`, `PolicyCheck`	Every decision with its reason and snapshot; deterministic action IDs; at most one open action per executor.
Chain ports	`Lease`, `EoaNonceReservation`, `TransactionAttempt`, `AttemptEvent`, `MonitorCursor`, `RiskEvent`	Fenced leases; idempotent nonce reservations; encrypted signed attempts with append-only history; the monitor cursor and canonical or orphaned risk events.
Operations	`Job`, `WorkerHeartbeat`, `ActivityEvent`	The queue; worker liveness and mode; the owner's append-only timeline.
Evaluation	`EvaluationRun`, `EvaluationCaseResult`	Benchmark runs with versions and per-persona results.

7.2 Concurrency rules

- Workflow changes and their follow-up jobs commit in one transaction; job keys make enqueueing idempotent.
- Jobs are claimed with `FOR UPDATE SKIP LOCKED` and a lease; a crashed worker's job is reclaimed.
- Signer and monitor work hold a row lease across RPC calls (never a database transaction); every write locks the lease row and checks its fencing token.
- Answers use optimistic concurrency on `Intent.version`; approvals compare-and-set on status and preview hash.
- Partial unique indexes enforce the invariants that matter most: one authorized mandate per executor, one open action per executor, one live attempt per executor action nonce.

> **The database is not the source of truth.** PostgreSQL holds workflow and history. Authority, funding, positions and receipts are always re-read from the chain before the worker acts.

CHAPTER 8

API and sessions

The API is a NestJS application under `/api/v1`. The machine-readable contract lives in `docs/openapi.json`; outside production, Swagger UI shows it at `/api/docs`. It never holds keys: everything that changes the chain is a transaction the owner signs.

8.1 Signing in with Ethereum

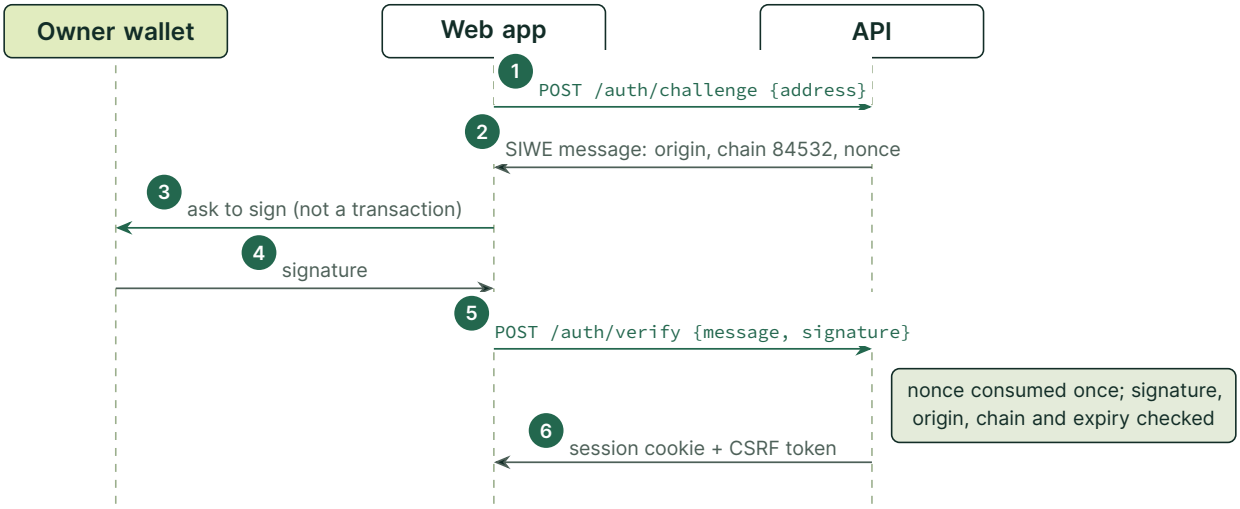


Figure 8.1: Signing in never approves a mandate or a transaction.

- The challenge is an EIP-4361 (SIWE) message bound to the web origin, chain 84532 and a single-use nonce that expires after 5 minutes. The web app refuses to ask for a signature if the message names another origin.
- Verification sets the `__Host-boundra_session` cookie (HttpOnly, Secure, SameSite=Strict, 12 hours) and returns a CSRF token.
- Every `POST` and `PATCH` needs the `Origin` header equal to `WEB_ORIGIN` and the `X-CSRF-Token` header.
- The owner is always taken from the session. Addresses in request bodies are never used for authorization, and another owner's resources return `404`.
- Bodies are limited to 16 KB; sign-in routes allow 10 requests a minute per client, other routes 120.

8.2 API tokens for apps and assistants

Apps built with the SDK and AI assistants using the MCP server sign in with an *API token* instead of the cookie:

1. `POST /auth/challenge {address, client: "token"}` returns a SIWE message whose statement tells the owner exactly what they grant: the token can read the account and prepare mandates for review, and cannot approve anything, sign transactions or move funds.
2. The owner's wallet signs it. Verification returns the token and sets no cookie. The signed statement decides the session kind, so an altered statement is rejected.
3. Clients send `Authorization: Bearer <token>`. No CSRF token is needed, since browsers never attach this header on their own, and CORS still admits only the web origin.

Tokens and cookies are never interchangeable, admin routes need the admin's browser session, and tokens are listed and revoked from the web app (`GET /auth/tokens`, `POST /auth/tokens/:id/revoke`, browser session only). Only token hashes are stored.

8.3 The owner journey as API calls

Step	Request	Notes
Public config	`GET /config/public`	Reviewed addresses, operator, vault descriptors.
Executor	wallet: `faucet.claim()`, `factory.createExecutor()`; then `POST /executors/register`	The server reads `executorOf(owner)`; the body is ignored.
Intent	`POST /intents {objective, amount, chainId}`	Rejects amounts above wallet plus executor balance.
Clarify	`POST /intents/:id/answers {questionId, alternativeId}`	One question at a time.
Draft	`POST /intents/:id/mandate-drafts {…}`	Tuple, preview, `previewHash`; optional `operator`.
Edit	`PATCH /mandate-drafts/:id`	Always a new revision.
Approve	`POST /mandate-drafts/:id/approve {previewHash}`	Returns the exact owner transactions.
Authorize	wallet: `setMandate`, `approve`, `fund`; then `POST …/authorization-check`	The worker sets `AUTHORIZED`.
Monitor	`GET /dashboard`, `GET /activity`	Confirmed balances, positions and timeline.
Control	wallet: `pause`, `resume`, `revoke`, `recoverIdle`, `ownerWithdraw`	The worker observes the change and stops.

8.4 Errors and health

Errors have one shape, with a stable `code`, a message and a recovery hint; internals are never echoed.

```
{ "error": { "code": "STALE_PREVIEW",
             "message": "...",
             "recovery": "Reload the latest preview and review it again." } }
```

Codes include `UNAUTHENTICATED`, `CSRF_REJECTED`, `RATE_LIMITED`, `VALIDATION_FAILED`, `CHALLENGE_INVALID`, `WRONG_CHAIN`, `INSUFFICIENT_BALANCE`, `STALE_PREVIEW`, `INVALID_STATE` and `DEPENDENCY_UNAVAILABLE`.

`GET /health/live` answers when the process is up; `GET /health/ready` also reports PostgreSQL, a fresh worker heartbeat and the worker's mode. The web app's `/status` combines this with the web release for monitoring.

CHAPTER 9

Web app

The web app (Next.js 16, React 19, wagmi 3, viem) is where owners do everything: sign in, set up their executor, describe a goal, answer ACCE's questions, review and approve the mandate, sign it into the contract, then watch and control the agent.

9.1 Pages

Route	What the owner does there
`/`	Landing page in the "Living Boundary" design.
`/app`	Sign in, one-time account setup (faucet, create executor), then the overview: funds, positions, controls and activity.
`/app/intents/[id]`	Answer clarifying questions one at a time, with "unsure" always available.
`/app/mandates/[id]`	Review the mandate in plain language, edit, approve, then authorize and fund it from the wallet.
`/app/risk`	Risk admin only: change a vault's risk state and capacity from the risk admin's wallet.
`/app/connect`	*Apps & AI*: create an API token (shown once, with a ready MCP configuration), see and revoke tokens.
`/status`	Same-origin readiness for monitoring: API, database, worker mode, releases.

9.2 Wallet transactions done honestly

Every owner transaction is built from the exact values the API returned (amounts as exact base units, never rounded) and goes through the same handling:

- The network guard keeps the wallet on chain 84532 and offers to add or switch the network.
- Before asking the wallet, the app simulates the call and sets its own gas limit, instead of trusting a wallet estimate made against a different node.
- States are shown honestly: waiting for the wallet, sent, confirmed, or reverted. A sent transaction is never shown as done until it is confirmed, and wallet errors are reported in plain language.
- Balances, positions and mandate status shown to the owner come from chain-confirmed state through the API.

9.3 Reviewing a mandate

The review screen is where consent happens, so it shows everything that will be enforced: the selected vault, every cap, the lockup, the expiry, the operator (marked “your own agent” when the owner named one), every notice and the testnet labels. Approval sends the `previewHash` of exactly that preview. Any edit produces a new revision that must be reviewed again.

> **Plain language, exact numbers.** Agent escalations, holds and refusals are explained in words an owner understands (for example why the agent is waiting, or why funds cannot leave a locked vault yet), while every amount stays exact. The activity timeline links to the block explorer only for real transactions.

9.4 Design

The visual language follows the approved “Living Boundary” references: a forest, sage and lime palette on ivory, the two-arc mark that holds a lime core (the owner’s intent), Plus Jakarta Sans for text and Newsreader for headlines. This document uses the same palette.

Forest	Green	Sage	Lime	Amber	Ivory
#122D27	#23674E	#9BAF8D	#A6C74C	#D8A13B	#F6F8F2

CHAPTER 10

Language model use

Boundra uses a language model for one narrow job: making text read naturally. It rewords a question ACCE has already chosen, or restates facts from a preview as a short summary. It never decides what is asked, which vault is used or what the limits are.

10.1 Providers

LLM_PROVIDER	Model	How it is called
disabled (default)	none	Reviewed templates, labelled TEMPLATE wherever they are shown.
openai	LLM_MODEL, required (for example gpt-4.1-mini)	Chat completions with strict structured output; low reasoning effort for reasoning models; refusals and content filtering count as a refusal.
anthropic	claude-opus-5-5 by default	Structured output, low effort, server-side refusal fallback.

Both providers share the same limits: a 20-second deadline for the whole call, at most three attempts, a cap on output tokens and an hourly token budget (settings LLM_DEADLINE_MS, LLM_MAX_OUTPUT_TOKENS and LLM_MAX_TOKENS_PER_HOUR). When the budget is spent, the template is used instead of spending more.

10.2 Validation and fallback

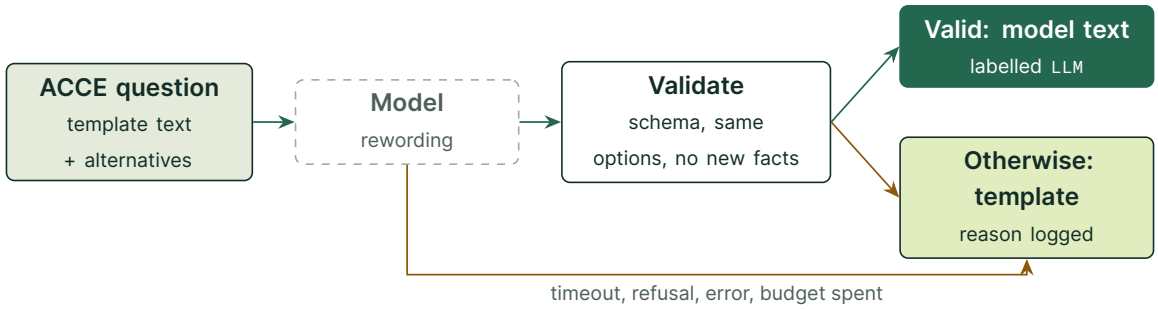


Figure 10.1: Every model output is checked; anything doubtful falls back to the reviewed template.

A reworded question must keep the same alternative IDs in the same order. Any output that adds numbers, vaults, protocols, addresses, links or guarantees is rejected. The fallback reason (NEW_NUMBER, ALTERNATIVES_CHANGED, TIMEOUT, REFUSAL and so on) is recorded in the activity log,

and the screen shows whether the text came from the model or the template.

Why this is safe. The model sees only the template question and its alternatives, or the preview's facts. It cannot change what the owner is asked, what they approve, or what the contract enforces. With the model disabled, Boundra works exactly the same.

Verified live. A real call from the API container of the Docker stack, with `gpt-4.1-mini`, reworded the first ACCE question and passed validation with no fallback.

CHAPTER 11

SDK and MCP server

Because the limits live in the owner's executor contract, the agent does not have to be Boundra's. Any program can be the operator and stay inside the same boundaries. Two packages make that practical: `@boundra/sdk` for developers and an MCP server for AI assistants such as Claude.

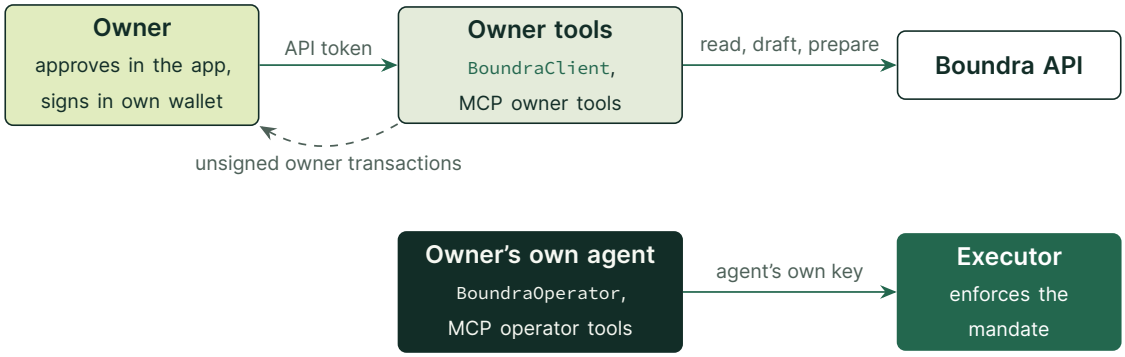


Figure 11.1: Owner tools never sign. The operator side signs only with the agent's own key, and the contract bounds it.

11.1 `@boundra/sdk`

Export	What it does
`BoundraClient`	The owner's API client: sign in with an API token, start a goal, answer questions, draft (optionally for the owner's own agent), approve, read the dashboard and activity.
`ownerAuthorizationCalls`, `ownerActionCall`, `faucetClaimCall`, `createExecutorCall`	Build *unsigned* transactions for the owner's wallet to sign.
`BoundraOperator`	The agent side, with the agent's own key: reads the confirmed mandate, explains what it may do per vault and why not, prepares the action with the same chain guard as the worker, simulates, then sends.
`mockUsdc()`	Exact amount helper (6 decimals).

An operator refusal has a stable code and a recovery hint (for example `WRONG_OPERATOR` when another agent's key is used), and it happens before anything is sent. The contract would reject it anyway.

11.2 MCP server

The MCP server (`apps/mcp`, stdio) gives AI assistants ten tools. Owner tools need an API token; operator tools need the agent's own key. Without either it offers only an overview with setup guidance.

Tool	Needs	What it does
`boundra_get_overview`	owner token	Executor, mandate, funds, positions, chain status.
`boundra_list_activity`	owner token	Timeline, with transaction links only for real transactions.
`boundra_get_mandate`	owner token	The exact limits in force and the notices the owner sees.
`boundra_start_goal`	owner token	Starts a goal and returns the first clarifying question.
`boundra_answer_question`	owner token	Records the owner's answer ("unsure" always allowed).
`boundra_draft_mandate`	owner token	Drafts or revises a mandate; the owner approves in the app.
`boundra_prepare_owner_action`	owner token	Pause, resume, revoke, recover, withdraw as an *unsigned* transaction.
`boundra_operator_status`	agent key	What a mandate lets this agent do now, and why not.
`boundra_deposit`, `boundra_withdraw`	agent key	Act within the mandate with the agent's own key.

Rules that do not bend

No tool approves a mandate or signs for the owner. The operator key is the agent's, never the owner's, deployer's or risk admin's. A stolen agent key is still bounded by the mandate, and the owner can pause or revoke at any time.

Verified end to end. A full-stack run on a local chain drove the whole journey through the SDK and MCP server: token sign-in, a mandate for the owner's own agent, the worker standing aside, the agent's deposit, another agent's key refused, a withdrawal paid to the owner, and a revoked token rejected.

CHAPTER 12

Security model

This chapter collects the security properties from the rest of the document in one place. It is a self-assessment for a controlled testnet prototype, not an independent audit or an approval for real funds.

12.1 Trust boundaries

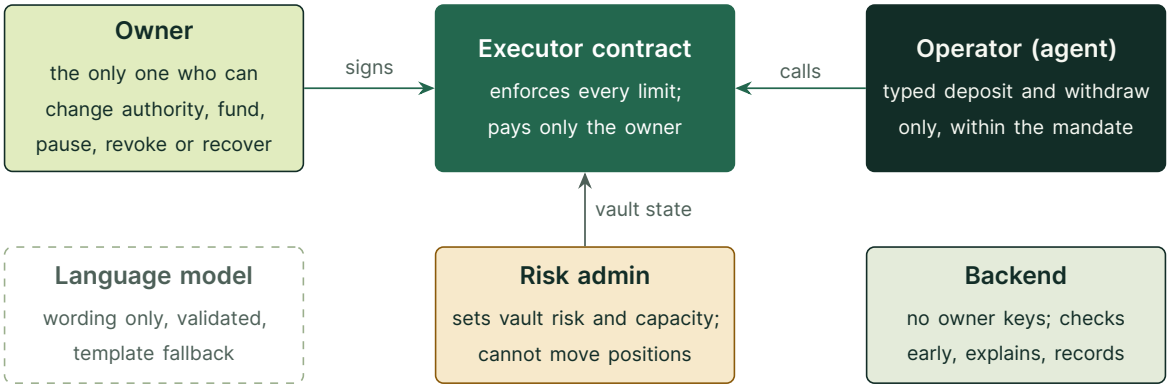


Figure 12.1: Each role has one job and no way to take over another's.

12.2 Threats and mitigations

Threat	Mitigation
Agent key stolen	The attacker can only make calls the mandate allows: capped, one allocation, allowed vaults, withdrawals to the owner. The owner pauses or revokes, recovers funds, and authorizes a new agent with a fresh mandate after settlement.
Agent's model or policy is wrong	Same bound as a stolen key. The policy is deterministic and tested, and `prepareAction` plus simulation refuse actions outside the mandate before sending.
Server compromised	No owner keys are on the server. The agent key is bounded as above. Secrets are runtime settings only, never in git or images.
Fake contracts or events	The backend trusts one reviewed manifest and checks runtime bytecode hashes, chain, asset and factory relationships. Risk events are accepted only from the two pinned vaults.
Replay or stale actions	Fresh action ID, current nonce and mandate version, and an exclusive deadline are checked on chain. Reverts consume neither nonce nor ID.

Threat	Mitigation
Owner approves something other than shown	Approval must present the `previewHash` of the exact preview; the worker authorizes only when the on-chain tuple matches that revision field by field.
Cross-site requests	First-party `__Host-` cookie with SameSite=Strict, exact `Origin` check and CSRF token on every state change; bearer tokens are never sent by browsers on their own.
Duplicate or lost transactions	Persist-before-broadcast, fenced leases, idempotent nonce reservations, identical-byte recovery and a 20-block reorg window.
Language model misleads	It cannot change questions, options, limits or approvals; outputs adding facts are rejected and the template is used.

12.3 Contract evidence

Foundry unit, fuzz and stateful invariant tests (`pnpm contracts:test`) cover owner-only changes, exact principal and decimals, the supply cap, lockup, bounded capacity, beneficiary routing, pause, revoke and expiry recovery, mandate replacement, per-action and cumulative budgets, target and action masks, wrong chain, version, nonce and deadline, replay, arbitrary-call denial and partial allocation and exit. The invariant tests run randomized sequences of deposits, withdrawals, pauses and recoveries and assert budget bounds, at most one allocation, owner-only proceeds, principal conservation, vault backing and zero leftover allowance.

> **Limits of this evidence.** Tests show the presence of the intended behaviour, not the absence of all bugs. An independent security review is still recommended before any wider use, and an audit before any real funds.

12.4 Operational rules

- Operator, deployer and risk admin are separate accounts. The worker refuses to run in `execute` mode if the operator equals the risk admin.
- Secrets (session secret, agent key, attempt encryption key, LLM key, database password) are runtime settings; `deploy/.env` is gitignored and excluded from Docker builds.
- Execution stays disabled until a trusted manifest, a funded operator, explicit gas and fee caps and the durable store are all in place.

CHAPTER 13

Testing and evidence

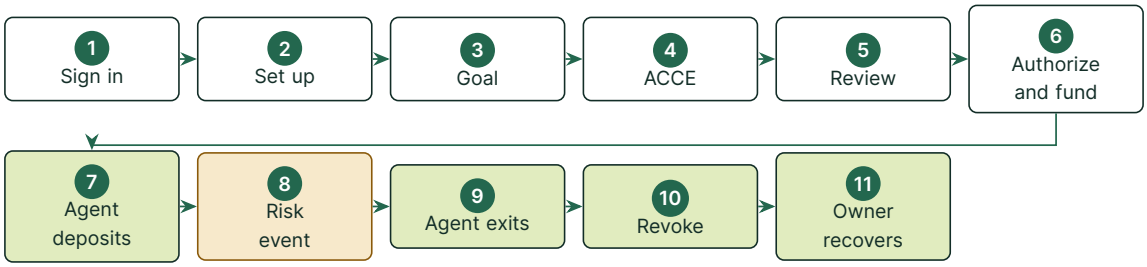
Every part has its own tests, and two suites run the whole system with nothing mocked. The counts below are from the last full local run.

13.1 Test suites

Suite	Tests	What it proves
Contracts (Foundry)	32	Unit, fuzz and stateful invariant tests of every rule in chapter 3.
ACCE	20	Question selection, scoring, narrowing, vault selection and the mandate builder.
Language model	23	Request shape, limits, validation and every fallback, for both providers.
API integration	24	Sessions, API tokens, intents, drafts, approvals and owner isolation against real PostgreSQL.
Worker integration	29	Stores, leases, crash recovery, concurrency and orchestration against real PostgreSQL.
Worker unit	19	Decisions, execution service and the risk monitor.
SDK unit / on chain	8 / 5	The client, and the operator against the real contracts on a local chain.
MCP server	8	Tool sets, refusals and configuration errors with a real MCP client.
Browser, mocked API	53	Every screen and owner transaction against the real contracts on a local chain.
Browser, real stack	2	The full journey with nothing mocked (below).
Worker full flow on chain	1 flow	Deposit, risk exit, escalation and revocation on a local chain.

13.2 The end-to-end journey

The real-stack browser suite starts the API and the worker (in `execute` mode) on a temporary database and a local chain, and runs:



It also runs a same-prompt walkthrough with two personas that answer differently and get different mandates, and a language-model outage that falls back to labelled templates without blocking anything.

13.3 Benchmark: B0–B3

The evaluation runner compares four ways of reaching a mandate. All use the same mandate builder and action set; only elicitation and consent differ.

Method	Elicitation	Consent
B0	One model call reads preferences from the goal text	The draft is the mandate
B1	Same one-shot draft	Static confirmation screen
B2	Five fixed questions: liquidity, lockup, leverage, amount, vaults	Same approval step as the product
B3	ACCE adaptive questions	Same approval step as the product

A *mismatch* is a mandate that breaks at least one of the persona's non-negotiable preferences. The fixture has 50 personas (20 development, 30 held out). Hidden preferences are read only by the simulator, a lint rule stops baselines from importing it, and the held-out split refuses to run until the fixture is frozen.

> **No results claimed yet.** The benchmark fixture is awaiting its label review and freeze, so no benchmark results are reported here. Results will be published only after a frozen, held-out run.

CHAPTER 14

Deployment and operations

Boundra ships as Docker images built from one Dockerfile, and can run in two ways: a Docker Compose stack on any server, or Render. Both use the same images and the same reviewed Base Sepolia manifest.

14.1 Docker Compose stack

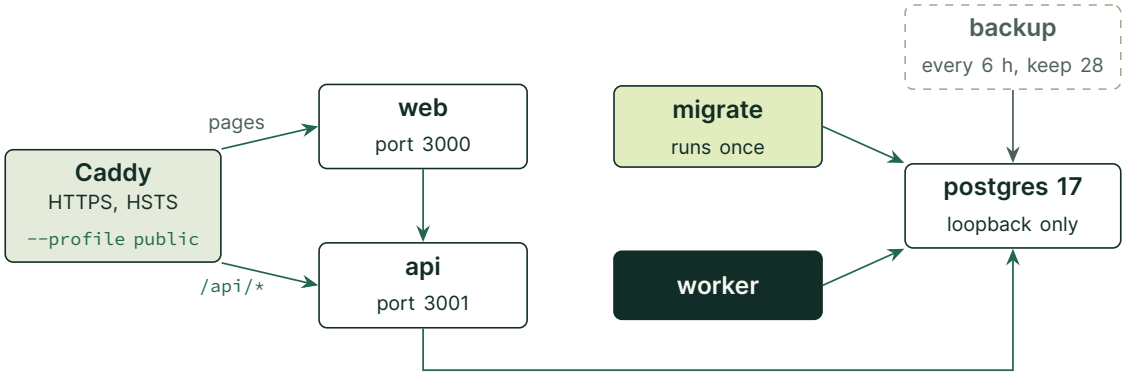


Figure 14.1: Start order is enforced: migrations finish before the worker starts, and the API waits for a healthy worker.

- Settings come from `deploy/.env` (created by `node scripts/setup.mjs --deploy`; secrets readable by the operator only). The manifest is mounted read-only from `deploy/manifests/`.
- With a public `APP_DOMAIN` whose DNS points at the host, Caddy obtains and renews Let's Encrypt certificates itself.
- Releases: tagging a green commit `v*` publishes `boundra-{web,api,worker,migrate}` images to GHCR. Each image carries its release identifier, which `/status` and the health endpoints report.
- A backup is taken before every upgrade; migrations only move forward, so a rollback that crosses a migration restores the backup.

14.2 Render, free plan

Render has no free plan for background workers or private services, so the Blueprint (`render.yaml`) runs everything in *one* free web service, with a free PostgreSQL database.

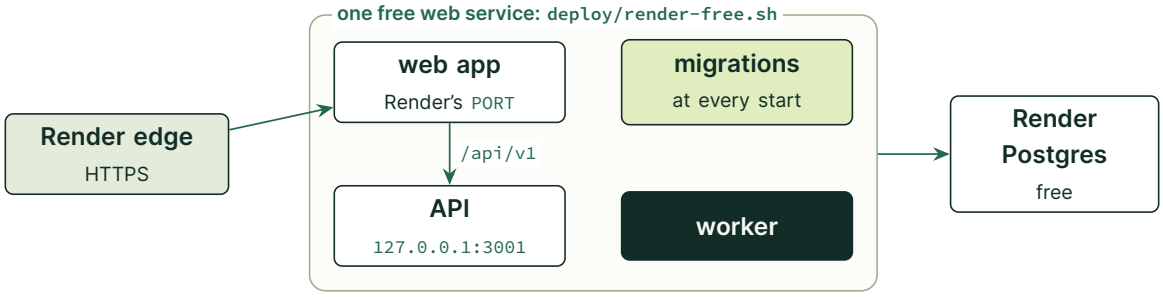


Figure 14.2: If any of the three processes exits, the container exits and Render restarts it.

Measured locally under the free limits	Result (512 MB memory, 0.1 CPU)
Start-up on a fresh database	About 2 minutes to ready, all 5 migrations applied
Memory in use	413 MB of 512 MB (heap caps per process)
Pages and assets	Served, including static files
A process crashes	The container exits so Render restarts it

Free-plan limits. A free service sleeps after about 15 minutes without visitors, and the agent sleeps with it; the next visit takes about a minute to wake it. Free instance hours are limited per month, and the free database expires after Render's free period. Switching the service and database to paid plans removes these limits without other changes.

14.3 Settings that matter

Setting	Purpose
`DEPLOYMENT_MANIFEST_PATH`	The reviewed manifest; without it the worker only observes.
`WEB_ORIGIN`	The exact public origin; sign-in messages and the Origin check depend on it.
`OPERATOR_ADDRESS`	Boundra's agent, public address only (API).
`OPERATOR_PRIVATE_KEY`	The agent's own key (worker only). Never the owner's, deployer's or risk admin's.
`EXECUTION_ENABLED`, `ATTEMPT_ENCRYPTION_KEY`, `MAX_GAS`, `MAX_FEE_PER_GAS_WEI`, `MAX_PRIORITY_FEE_PER_GAS_WEI`	Required for `execute` mode. Fee caps are ceilings: above them the worker skips the action rather than overpay.
`MONITOR_BATCH_BLOCKS`	Risk-scan size (180 fits the public RPC).
`ADMIN_ADDRESSES`	Wallets allowed to use admin routes (lowercase).
`LLM_PROVIDER`, `LLM_MODEL`, `LLM_API_KEY`	Optional wording provider (chapter 10).
`TRUST_PROXY`	True only behind a trusted proxy (Caddy or Render).

14.4 Checking a deployment

1. `/status` reports `"status": "ok"` and `"worker": "execute"`; `monitor_only` means a worker setting is missing or rejected, and the logs name which (never its value).
2. `/api/v1/health/ready` reports the database as connected and a fresh worker heartbeat.
3. The worker logs show `Worker mode: execute` and no repeated monitor failures. On first start the risk monitor needs about 40 minutes to catch up from the deployment block.

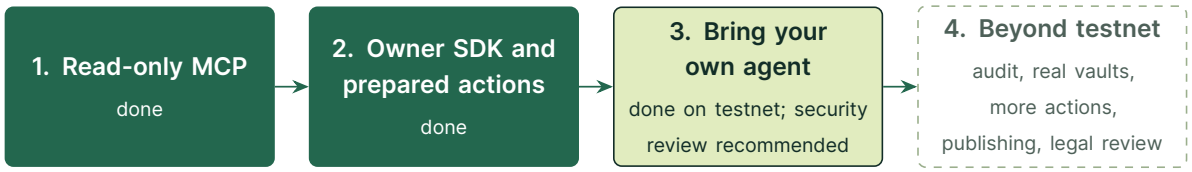
CHAPTER 15

Limitations and roadmap

15.1 Known limitations

Limitation	Why, and what it means
Testnet only	Base Sepolia, MockUSDC and two controlled mock vaults. Illustrative yields do not accrue; principal stays 1:1.
Small action set	Two vaults and one deposit allocation per mandate. Deliberately narrow, so the guarantees are easy to check.
Risk events are a fixture	A risk admin sets vault risk by hand to stand in for a real incident; there is no oracle.
No fee replacement	A stuck transaction is reconciled or released after its deadline; fees are never bumped automatically.
Benchmark not yet run	The persona fixture awaits label review and freeze; no results are claimed.
Packages not published	The SDK and MCP server are built and tested but not on npm; publishing needs a licence decision.
Free hosting sleeps	On Render's free plan the agent sleeps with the service when nobody visits.

15.2 Roadmap



Before any real funds

An independent audit of the contracts, integrations with real, reviewed vaults, a review of the agent's policy for real market conditions, and a legal review. Until then, Boundra stays on testnet.

CHAPTER A

Glossary

Term	Meaning
ACCE	Action-Conditioned Counterfactual Elicitation: asks only questions whose answer could change the agent's allowed action.
Action ID	A unique identifier for one agent action; the executor accepts each only once.
Allocation	The single deposit a mandate allows; a partial deposit also completes it.
Base Sepolia	Base's public test network (chain 84532).
Confirmation depth	How many blocks deep state must be before it is trusted (5 by default).
DecisionFlip	How often an answer to one dimension changes the selected action, over all combinations of the others.
Escalation	The agent stops and tells the owner, because no allowed action fits.
Executor	The owner's own contract: holds funds, stores the mandate, checks every agent call.
Fencing token	A number that grows with every new lease, so an outdated lease holder cannot write.
Mandate	The twelve-field set of limits the owner signs into their executor.
Manifest	The reviewed list of contract addresses and code hashes the backend trusts.
MCP	Model Context Protocol, the standard AI assistants use to call tools.
MockUSDC	The test stablecoin (6 decimals) used instead of real money.
Operator	The agent allowed by a mandate to call `executeDeposit` and `executeWithdraw`.
Preview hash	keccak256 of the canonical preview; approval must present it.
Reorg	A chain reorganisation, where recent blocks are replaced; handled by confirmation depth, overlap and re-checks.
Risk admin	The account that sets the mock vaults' risk state for the demo.
SIWE	Sign-In with Ethereum (EIP-4361): proving wallet ownership by signing a message.

CHAPTER B

Where to find things

Topic	Source
Contract rules	`docs/contracts.md`, `docs/security.md`, `contracts/src/`
Chain integration	`docs/chain-integration.md`, `packages/chain`
ACCE and agent policy	`docs/acce.md`, `packages/acce`, `apps/worker/src/orchestration/decision.ts`
Execution and monitoring	`docs/worker-chain-port.md`, `apps/worker/src/chain/`
Data model	`docs/data-model.md`, `packages/database/prisma/schema.prisma`
API	`docs/api.md`, `docs/openapi.json`
SDK and MCP	`docs/roadmap-sdk-mcp.md`, `packages/sdk`, `apps/mcp`
Operations	`docs/runbook.md`, `docs/deploy-render.md`, `compose.yaml`, `render.yaml`
Benchmark	`evaluation/README.md`